

ROB 550 Armlab Report

Pranav Jain, Junwoon Lee
{psjain, junwoon}@umich.edu

Abstract—The ability of robots to operate autonomously in real-world environments depends fundamentally on their perception, path planning, and control capabilities. These functions require not only robust motion algorithms but also sophisticated sensing systems that enable the robot to detect, reason, and act based on its observations. This work presents an integrated vision-based manipulation framework to enhance the autonomy of the RX200 robotic arm in object pick and place tasks. Using an RealSense L515 RGB-D camera, both color and depth data are acquired to sense the workspace. Through OpenCV-based image processing, the system extracts the 3D position, orientation, color, and size of blocks within the arm’s operational range. These are used to solve a numerical inverse kinematics based on forward kinematics (Denavit–Hartenberg form) and for, allowing the arm to precisely reach and manipulate target objects. The proposed system demonstrates the ability to detect, sort, stack, and place colored blocks autonomously in competitive lab environments.

I. INTRODUCTION

Robotic manipulators have become indispensable in fields that require precision, reliability, and repeatability, including industrial automation, medical surgery, and space exploration. Their ability to perform complex and repetitive tasks with minimal error has made them essential in situations where accuracy and safety are critical.

This project explores the development of an autonomous manipulation system using the ReactorX-200 robotic arm and the Intel RealSense L515 LiDAR depth camera. The RX-200 is a five-degree-of-freedom manipulator consisting of a waist, shoulder, elbow, wrist pitch, and wrist rotation joint, and it is equipped with a gripper capable of handling small lightweight objects. The L515 camera provides both RGB and depth information using a solid-state LiDAR sensor, offering high-precision three-dimensional measurements within several meters of range. Together, these components form a system capable of perceiving and interacting with objects in a defined workspace. The implementation was carried out within the Robot Operating System (ROS) framework using the Python programming language. The system was designed to identify, grasp, and place colored blocks using conventional computer vision techniques, forward and inverse kinematics, and numerical inverse kinematics. The camera was calibrated to the workspace through AprilTag markers in terms of extrinsic calibration and

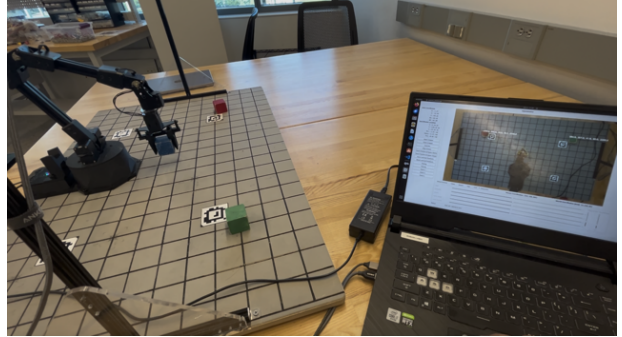


Fig. 1. Proposed Deployment Overview: The system utilizes both depth and RGB data from the Intel RealSense L515 camera to detect the target blocks, after which the RX-200 robotic arm performs the corresponding manipulation tasks using inverse kinematics for precise motion control.

checker board in terms of intrinsic calibration, enabling accurate transformation between the image and robot coordinate frames.

The objective of this project was to implement a complete perception-to-manipulation pipeline for the RX-200 robotic arm as shown in Fig. 1. The system successfully performed tasks such as detecting, sorting, stacking, and relocating colored blocks within its workspace. Although occasional limitations were observed in the robustness of the inverse kinematics solver, particularly in edge cases, the overall performance remained stable and reliable throughout testing and competition. These results demonstrate the effectiveness of our vision-guided robotic manipulation system for constrained real-world applications.

II. METHODOLOGY

A. Intrinsic Camera Calibration

The intrinsic calibration determines the internal parameters of a camera that describe the mapping between the three-dimensional camera coordinate frame and the two-dimensional image plane. These parameters include the focal lengths (f_x, f_y) and the center of lens (c_x, c_y) , which together form the intrinsic matrix $\mathbf{K}$:

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \quad (1)$$

This matrix defines a projective transformation from 3D camera coordinates $[X_c, Y_c, Z_c]^T$ to 2D image coordinates $[u, v]^T$ through the pinhole camera model:

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \begin{bmatrix} I & | & 0 \end{bmatrix} \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix}, \quad (2)$$

where λ is a scale factor corresponding to depth.

For the Intel RealSense L515, the intrinsic parameters were obtained using the ROS camera calibration tool and an 8×6 checkerboard pattern. The checkerboard was moved to various positions and orientations across the camera's field of view to ensure that the calibration covered a broad range of viewing angles and depths. To improve accuracy, the calibration was repeated five times, and the mean result was adopted as the final intrinsic matrix.

The resulting intrinsic matrix of the RealSense L515 camera is:

$$\mathbf{K} = \begin{bmatrix} 927.73 & 0 & 618.42 \\ 0 & 926.72 & 370.89 \\ 0 & 0 & 1 \end{bmatrix}. \quad (3)$$

The obtained matrix was later used to convert 2D pixel coordinates into 3D camera-frame points, which served as input for object localization and inverse kinematics computation in the perception-to-manipulation pipeline.

B. Extrinsic Camera Calibration

Accurate 3D workspace reconstruction requires the camera's extrinsic parameters, which map points from the world frame to the camera frame via a rigid transformation composed of a rotation $\mathbf{R} \in SO(3)$ and a translation $\mathbf{t} \in \mathbb{R}^3$:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \mathbf{R} \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix} + \mathbf{t}. \quad (4)$$

$$\mathbf{H} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} \in SE(3). \quad (5)$$

While coarse estimates can be obtained from tape-measure distances and rough orientation readings, we employ a vision-based procedure for precision. Four AprilTags are affixed to the calibration board at known world coordinates; their corresponding image coordinates are collected from the RGB stream. Given these 2D–3D correspondences and the known intrinsics $\mathbf{K}$, we estimate $(\mathbf{R}, \mathbf{t})$ using OpenCV [1] Perspective- n -Point solver (`cv2.solvePnP`) with the `SOLVEPNP_IPPE`

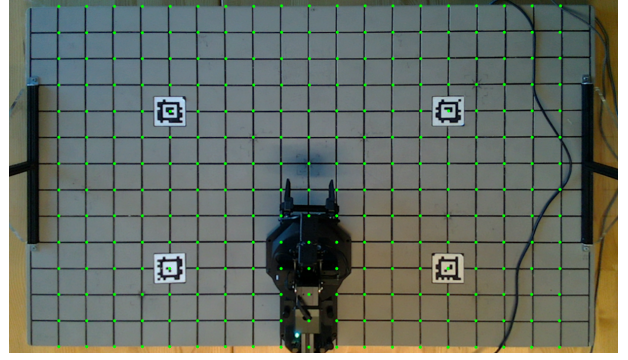


Fig. 2. Grid projection used to verify the accuracy of the extrinsic calibration.

flag, which exploits the fact that all AprilTags lie on a plane. The returned rotation vector is converted to a rotation matrix via Rodrigues' formula to form $\mathbf{R}$, which we combine with $\mathbf{t}$ to obtain $\mathbf{H} \in SE(3)$.

With depth $d = Z_c(u, v)$ available from the L515, a pixel $\mathbf{u} = [u \ v \ 1]^T$ is first back-projected into the camera frame using the inverse intrinsics:

$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = d \mathbf{K}^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}. \quad (6)$$

Expressing $\mathbf{X}_c$ in homogeneous form and applying the inverse extrinsics yields the world coordinates:

$$\begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} = \mathbf{H}^{-1} \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix}. \quad (7)$$

Because the RGB sensor and the LiDAR depth sensor have slightly different optical centers, we observed drift across the board when fusing RGB detections with depth. We resolve this issue by applying a small rigid 2D alignment between the RGB and depth images (in-plane rotation and translation). The final extrinsic matrix is:

$$\mathbf{H} = \begin{bmatrix} 0.999 & 0.0108 & -0.0192 & -2.33 \\ 0.0143 & -0.981 & 0.192 & 164 \\ -0.0168 & -0.192 & -0.981 & 1050 \\ 0.00 & 0.00 & 0.00 & 1.00 \end{bmatrix}. \quad (8)$$

Finally, to obtain a convenient region of interest that provides a rectified, top-down view of the workspace, we compute a planar homography $\mathbf{H}_{\text{img}} \in \mathbb{R}^{3 \times 3}$ from the four board corners, (350, 540), (900, 540), (900, 230), and (350, 230). And apply a `warpPerspective` function of OpenCV to the RGB image. $\mathbf{H}_{\text{img}}$ is as follow:

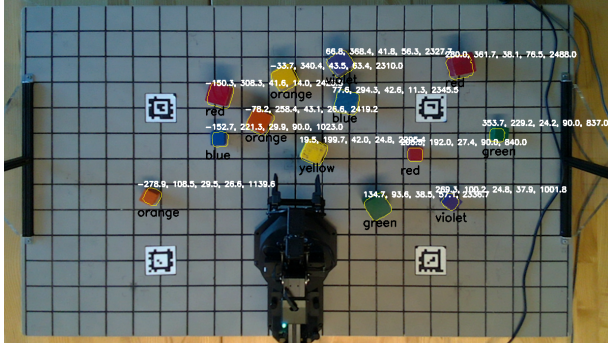


Fig. 3. Block Detection Example.

$$\mathbf{H}_{\text{img}} = \begin{bmatrix} 1.12 & -0.0964 & -74.1 \\ -0.0132 & 1.0098 & -48.5 \\ 0.0 & 0.0 & 1.0 \end{bmatrix}. \quad (9)$$

We expose an automated calibration routine in the GUI that (i) detects the four AprilTags, (ii) solve PnP problem using the four points, and (iii) constructs $\mathbf{H}$. Validation is performed by projecting a world-grid overlay into the GUI, as shown in Fig. 2.

C. Block Detection

The goal of the detector is to recover, for each block, its 3D position, in-plane orientation, size, and discrete color label, as shown in Fig. 3. The detector directly uses the RGB frame and the raw depth frame, calibrated by the camera intrinsics, extrinsics, and a display homography.

For each pixel (u, v) , homogeneous image coordinates are formed and back-projected to the camera frame via $\mathbf{K}^{-1}$, scaled by the per-pixel depth. The resulting 3D camera points are lifted to homogeneous coordinates and transformed into the world frame using $\mathbf{H}^{-1}$. These world coordinates are reshaped to the image layout. Here, the world Z component defines a depth map in the display space. Both the depth map and the per-pixel world coordinates are then warped with the precomputed homography to align with the GUI view. A workspace mask excludes the robot arm and any regions outside the board, after which candidate block surfaces are segmented from the height map.

Contours are extracted from this binary segmentation using OpenCV. Each contour is validated and refined prior to visualization and using for down-stream tasks. The point set is clipped to image bounds. Here, contours with fewer than three points are discarded. A robust depth reference z_{ref} is computed as the median of the largest up-to 20 depth samples within the contour. Image moments provide the centroid (u_c, v_c) . To reduce

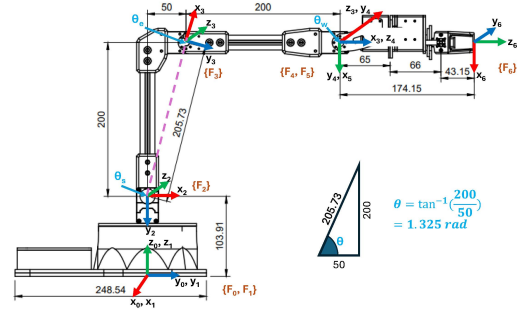


Fig. 4. Forward Kinematics Frames with Angles Used for Inverse Kinematics.

TABLE I
DENAIVT-HARTENBERG PARAMETERS OF THE MANIPULATOR

Link	θ_i [rad]	d_i [mm]	a_i [mm]	α_i [rad]
0	0	0	0	0
1	$\theta_1 + \pi/2$	103.91	0	$-\pi/2$
2	$\theta_2 - 1.325$	0	205.73	0
3	$\theta_3 + 1.325$	0	200	0
4	$\theta_4 + \pi/2$	0	0	$\pi/2$
5	θ_5	174.15	0	0

false detections, contours with area < 10 are dropped. In addition, a filled contour mask is used to collect interior depth values; if the interior median is more than approximately 5% below z_{ref} , the region is rejected to avoid hollow or recessed surfaces.

For accepted contours, Color is assigned directly in RGB by grouping the contour into a mask, computing the mean RGB over the masked pixels, and selecting the nearest distance from a fixed dictionary of labeled RGB samples.

D. Forward Kinematics

We solve forward kinematics (FK) using the Denavit–Hartenberg (DH) parameters [2]. To calculate the DH parameters, we apply a frame to each joint of the arm, following the DH convention, as shown in Fig. 4. Using these frames, we calculate the DH parameters as shown in Table I. Then, we calculate the transformation matrix for each link using trans_i and multiply them together to get the FK of the arm as a function of theta $\theta = (\theta_1, \theta_2, \theta_3, \theta_4, \theta_5)$, where each θ_i is the angle of the joints. trans_i is calculated using:

$$\text{trans}_i(\boldsymbol{\theta}, \mathbf{d}, \mathbf{a}, \alpha) = \begin{bmatrix} \cos \theta & -\sin \theta \cos \alpha & \sin \theta \sin \alpha & a \cos \theta \\ \sin \theta & \cos \theta \cos \alpha & -\cos \theta \sin \alpha & a \sin \theta \\ 0 & \sin \alpha & \cos \alpha & d \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (10)$$

After constructing each transformation matrix trans_i as in Eq. (10), the complete forward kinematics (FK) of the manipulator can be obtained by sequentially multiplying the individual homogeneous transformation matrices. The overall transformation from the base to the end-effector frame is expressed as:

$$\mathbf{T}_0^n(\boldsymbol{\theta}) = \mathbf{A}_1(\theta_1) \mathbf{A}_2(\theta_2) \cdots \mathbf{A}_n(\theta_n), \quad (11)$$

where $\mathbf{A}_i$ corresponds to the homogeneous transformation matrix of the i th link, defined by the DH parameters of Table I.

Each $\mathbf{A}_i$ can be expressed as the product of fundamental rotations and translations:

$$\mathbf{A}_i = \text{Rot}_z(\theta_i) \text{Trans}_z(d_i) \text{Trans}_x(a_i) \text{Rot}_x(\alpha_i). \quad (12)$$

Consequently, $\mathbf{T}_0^n$ can be decomposed into a rotation matrix $\mathbf{R}_0^n$ and a translation vector $\mathbf{o}_0^n$ as:

$$\mathbf{T}_0^n = \begin{bmatrix} \mathbf{R}_0^n & \mathbf{o}_0^n \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}. \quad (13)$$

From $\mathbf{T}_0^n$, the end-effector position and orientation can be derived. The position is given by the translation vector $\mathbf{o}_0^n = [x \ y \ z]^\top$, while the orientation can be represented using the Z-Y-Z Euler angle convention:

$$\mathbf{R}_{YZZ} = R_z(\phi) R_y(\theta) R_z(\psi), \quad (14)$$

where (ϕ, θ, ψ) denote the angle for each axis. In practice, the complete FK equations allow the determination of the end-effector pose for any given set of joint angles $\boldsymbol{\theta} = (\theta_1, \theta_2, \theta_3, \theta_4, \theta_5)$. The resulting position and orientation were verified experimentally by comparing the computed pose against manually measured coordinates. Small deviations on the order of a few millimeters were observed due to mechanical backlash and gravitational effects on the joints, yet the FK results were within an acceptable accuracy range for the manipulator's workspace.

E. Inverse Kinematics

We solve inverse kinematics (IK) numerically. Given a target pose $\mathbf{x}_d = (x_d, y_d, z_d, \phi, \theta)$, we define the planar radius $\rho_d = \sqrt{x_d^2 + y_d^2}$ and the desired end-effector pitch $\psi_d = \text{pitch}$. And link lengths $[\ell_0, \ell_1, \ell_2, \ell_3]$ are:

$$\begin{bmatrix} \ell_0 = 103.91 \\ \ell_1 = 206.16 \\ \ell_2 = 200.00 \\ \ell_3 = 174.15. \end{bmatrix} \quad (15)$$

Where the objective joint triplet be $\boldsymbol{\theta} = [\theta_s, \theta_e, \theta_w]^\top$, the forward kinematics is:

$$\psi(\boldsymbol{\theta}) = \theta_s + \theta_e + \theta_w. \quad (16)$$

$$\begin{aligned} \rho(\boldsymbol{\theta}) &= \ell_1 \cos(-\theta_s + \alpha) \\ &+ \ell_2 \cos(-(\theta_s + \theta_e)) \\ &+ \ell_3 \cos(-(\theta_s + \theta_e + \theta_w)) \end{aligned} \quad (17)$$

$$\begin{aligned} z(\boldsymbol{\theta}) &= \ell_0 \\ &+ \ell_1 \sin(-\theta_s + \alpha) \\ &+ \ell_2 \sin(-(\theta_s + \theta_e)) \\ &+ \ell_3 \sin(-(\theta_s + \theta_e + \theta_w)) \end{aligned} \quad (18)$$

Here, a fixed geometric offset $\alpha = \arctan(4)$ is applied at the shoulder. Then, We form a three-equation residual.

$$\mathbf{r}(\boldsymbol{\theta}) = \begin{bmatrix} \rho(\boldsymbol{\theta}) - \rho_d \\ z(\boldsymbol{\theta}) - z_d \\ \psi(\boldsymbol{\theta}) - \psi_d \end{bmatrix}. \quad (19)$$

The solution is obtained with a Jacobian-free nonlinear solver:

$$\boldsymbol{\theta}^* = \text{fsolve}(\mathbf{r}, \boldsymbol{\theta}_0), \quad (20)$$

where $\boldsymbol{\theta}_0$ is the initial guess passed to `fsolve` of SciPy.

The base yaw is set analytically from the target XY location,

$$\theta_1 = -\text{atan2}(x_d, y_d), \quad (21)$$

and the wrist roll is assigned as a simple alignment

$$\theta_5 = \theta_1 + \phi. \quad (22)$$

Note that if any of the angles $\{\theta_1, \theta_s, \theta_e, \theta_w, \theta_5\}$ lies outside a permissible range (e.g., exceeding π in magnitude), the target pose is slightly perturbed and the numerical solve is repeated. This pragmatic fallback improves robustness in edge cases without altering the nominal solution when the initial target is feasible. All equations above are a direct transcription of our implementation:

- EQ_numerical_IK (389–412): Eqs. (16)–(18).
- IK_numerical (415–435): Eqs. (20)–(22).

F. Motion Planning

The motion planner generates pick-and-place trajectories from user clicks or autonomous detections. Plans are expressed as sequences of Cartesian waypoints

with fixed clearance margins, converted online to joint targets via the numerical IK described in the previous subsection. Execution is also synchronized with gripper actions.

For user interactive operation, the user clicks a pixel (u, v) on the GUI. The corresponding depth d is read from the raw depth frame, and the triplet (u, v, d) is mapped to world coordinates by $\mathbf{x}_w = \mathbf{H}^{-1}\mathbf{K}^{-1}[u \ v \ 1]^T d$ with the display homography applied consistently. Then, Cartesian waypoints are converted to joint targets with `IK_numerical`.

We use a small set of reusable functions: (i) *approach* above the goal by a fixed vertical clearance, (ii) *descend* to a grasp/place height, (iii) *grasp/release* with the end-effector aligned, (iv) *retreat* back to the clearance height, (v) *reset* to a initialized pose.

a) *Pick trajectory*: The planner builds an approach–grasp sequence at the selected world position:

- 1) **Approach**: move to $(x_w, y_w, z_w + 50, \theta = \frac{\pi}{2})$.
- 2) **Descend**: move to $(x_w, y_w, z_w, \frac{\pi}{2})$.
- 3) **Grasp**: close the gripper to secure the block.
- 4) **Retreat**: move to $(x_w, y_w, z_w + 50, \frac{\pi}{2})$.
- 5) **Reset**: return to a initialized configuration.

b) *Place trajectory*: The placement plan mirrors the pick sequence, but a larger vertical margin to improve clearance during transport:

- 1) **Approach**: move to $(x_w, y_w, z_w + 150, \theta = \frac{\pi}{2})$.
- 2) **Descend**: move to $(x_w, y_w, z_w, \frac{\pi}{2})$.
- 3) **Release**: open the gripper to place the block.
- 4) **Retreat**: move to $(x_w, y_w, z_w + 150, \frac{\pi}{2})$.
- 5) **Reset**: return to a initialized configuration.

III. RESULTS

A. Camera Calibration

The intrinsic parameters as calculated in Eq. (3) were compared with the factory-provided calibration values. The factory-provided intrinsic calibration is as follow:

$$\mathbf{K}_{\text{factory}} = \begin{bmatrix} 912.49 & 0 & 635.95 \\ 0 & 912.59 & 375.48 \\ 0 & 0 & 1 \end{bmatrix}. \quad (23)$$

As a result, the error between factory and our calibration were observed, which can be attributed to environmental factors such as temperature and curved calibration board used during calibration process. The curvature can introduce a non-linear error, especially near the edges of board.

The extrinsic calibration was performed to determine the relationship between the world coordinate system and the camera coordinate system. The extrinsic matrix from hand-crafted methods is as follow:

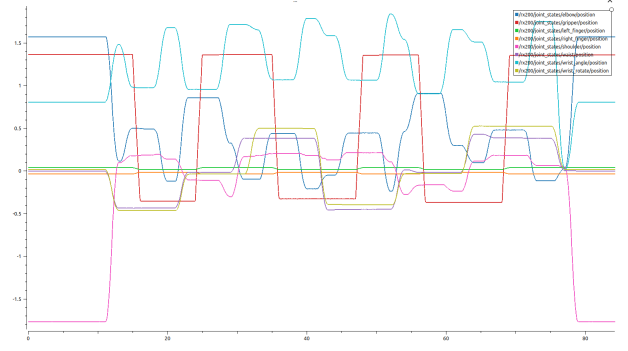


Fig. 5. Joint angles versus time during one teach-and-repeat cycle.

$$\mathbf{H}_{\text{hand}} = \begin{bmatrix} 1.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & -0.985 & -0.174 & 325 \\ 0.0 & 0.174 & -0.985 & 990 \\ 0.00 & 0.00 & 0.00 & 1.00 \end{bmatrix}. \quad (24)$$

For better calibration, we utilized an AprilTag-based approach combined with a PnP solver as solved in Eq. (8), which achieved higher accuracy compared to hand-crafted methods that rely on manual measurements of rotation and translation. As shown in Fig. 2, the calibration error was small. However, in terms of depth estimation, small errors remained due to the physical misalignment between the RGB and depth sensors. To deal with this issue, an additional offset correction was applied to compensate for the roll–pitch–yaw and translational differences between the two sensors.

B. Teach and Repeat

The “teach-and-repeat” enables the manipulator to reproduce a demonstrated sequence of actions. During the teaching phase, the robot arm was manually guided to a series of target poses, with corresponding joint angles and positions recorded as waypoints. These waypoints define a task trajectory that the robot autonomously executes in the repeat phase. The process involves three major steps in one full cycle: (1) picking and placing Block A to an intermediate position along Path 1, (2) moving Block B to the original position of Block A along Path 2, and (3) returning Block A to the initial position of Block B through Path 3.

During testing, each cycle produced slight deviations between commanded and actual end-effector coordinates due to cumulative pose errors and joint backlash. Figure 5 shows the evolution of five joint angles over time for one complete cycle.

Although the system could successfully execute approximately two full cycles before significant positional drift occurred, these inaccuracies arose primarily from

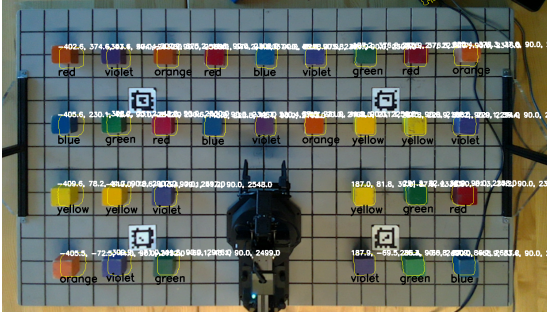


Fig. 6. Configuration used to evaluate the block detection method.

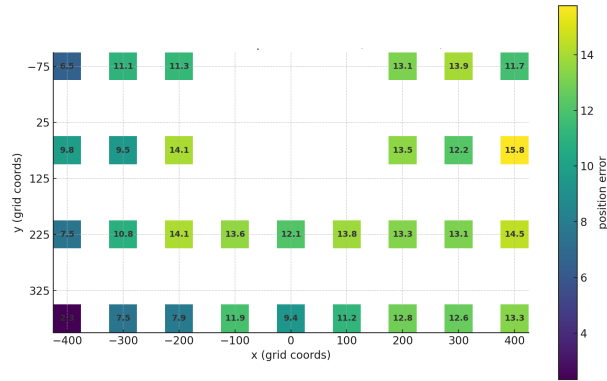


Fig. 7. Block detection heatmap showing localization errors across the workspace. The metrics are measured in mm.

the absence of camera feedback and calibration corrections. Despite this, the experiment effectively demonstrated waypoint-based motion planning and the reproducibility of predefined joint trajectories.

C. Block Detection

For autonomous block manipulation, color-based block detection was implemented using an RGB camera, as shown in Fig. 3. To evaluate the detection accuracy, blocks were placed at predefined locations, as shown in Fig. 6.

The detection errors were visualized as a heatmap, as shown in Fig. 7. The proposed method achieved a maximum position error of 15.8 mm for block localization. The system achieved near-perfect color classification across multiple trials. The remaining positional errors were mainly caused by camera calibration inaccuracies and misalignments between the RGB and depth sensors, which were partially compensated by applying the offset.

D. Competition

During the competition, three main tasks were conducted to evaluate the performance of the manipulation system.



Fig. 8. Event 1: Stacking 3 large RGB blocks.



Fig. 9. Event 2: Sorting 6 large blocks in a line in rainbow configuration.

Event 1: The robot successfully grasped three large RGB blocks and stacked them in predefined regions following the specified color order, as shown in Fig. 8.

Event 2: The system accurately aligned all six large blocks in the correct color sequence with only minimal positional and orientation errors, as shown in Fig. 9.

Event 3: The robot successfully stacked four large blocks; however, this task showed a limitation in the numerical inverse kinematics solver, as the manipulator had a problem in a singularity near certain joint configurations, as shown in Fig. 10.

IV. DISCUSSION

This work connected vision, calibration, kinematics, and a short sequence of motion steps into a single pipeline for reliable tabletop manipulation. The general

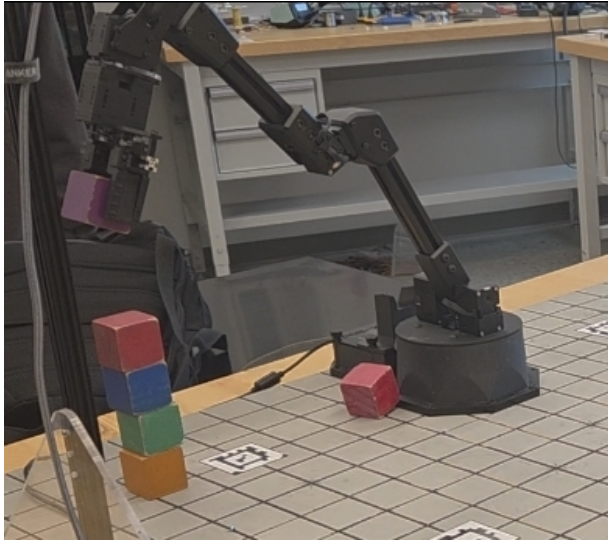


Fig. 10. Event 3: Stacking 4 large blocks.

idea was straightforward: use the RGB-D camera to detect a block, convert pixels and depth into a 3D point in the robot frame, solve inverse kinematics, and execute a hard coded approaches for grasp–retreat task.

On the sensing side, although intrinsic calibration with a checkerboard gave a roughly valid intrinsic matrix, our calibration board was slightly curved, which gives us systematic errors. As a result, the estimated parameters were not fully reliable. In addition, distortion was not explicitly corrected, so the back–projection from image to camera coordinates showed small spatial shifts. For the extrinsic calibration between the world and the camera, we used four AprilTags with a PnP solver. While this approach was simple and fast, it ignored depth information and only relied on 2D correspondences. Consequently, even though we could obtain depth data, the PnP result could not properly account for the true 3D alignment, which can limits the depth consistency for our final results of estimated coordinates.

The main error of estimating world coordinates from image came from the offset between the RGB camera and the depth sensor, which do not share the same optical center. When fusing color detections with depth measurements, a height error was shown, especially toward the edges of the board. We tried to reduce this effect by applying a rotational and translational offset between the two sensors and validating the result with a projected grid, although small residual errors remained. For single-block picking, this residual was acceptable, but for precise stacking, even a few millimeters of misalignment caused failure. So, we plan to perform separate calibrations for each sensing pair to reduce depth-related error.

For detection, we worked directly with the raw RGB frames to assign color labels and used the depth images to obtain geometric information. Using raw RGB is simple to implement and fast to compute. However, raw RGB can be sensitive to lighting conditions, for example, shadows from the gripper or uneven illumination can cause to shift the measured color closer to a neighboring class. Switching to the HSV or CIELAB color space would be helpful, as these representations separate brightness from chromatic components and can reduce the impact of lighting variations.

For motion planning, each pick or place action followed a four-step sequence: approach from above, descend to the target height, open or close the gripper, and retreat to a safe height. Although not time-optimal, this strategy avoided collisions. Inverse kinematics was solved numerically and worked well for normal configurations, but performance degraded near singularities. When a singular pose was given, the solver slightly perturbed the target to continue convergence, but this reduced the positional accuracy. To deal with this limitation, we plan to use a geometric and analytic inverse kinematics method, which can handle singular configurations more robustly and provide more stable solutions for manipulation tasks.

Overall, there are three key lessons what we learned from this project. First, reducing even a few millimeters of sensing error caused by intrinsic and extrinsic calibration can improve stacking accuracy. Second, consistent color detection was essential for stable operation. Working directly with raw RGB was simple and fast but highly sensitive to lighting variations, such as shadows or uneven illumination. Adopting a different color space that separates brightness from chromaticity, can make color detection more robust. Third, while the numerical inverse kinematics solver handled most configurations successfully, it showed unstable results near singularities. Introducing a analytic inverse kinematics formulation can improve stability.

V. CONCLUSION

This work presented an vision-based manipulation system for the RX-200 robotic arm, combining perception, calibration, kinematics, and motion planning. Using an RGB-D camera and numerical inverse kinematics, the robot was able to autonomously detect, grasp, and stack colored blocks in a tabletop environment. The system demonstrated stable performance during competition tasks, completing multiple pick-and-place operations with fair accuracy and robustness.

Future works will focus on refining calibration between the RGB and depth sensors and incorporating geometric inverse kinematic for more complex autonomous manipulation tasks.

REFERENCES

- [1] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [2] J. Denavit and R. S. Hartenberg, "A kinematic notation for lower-pair mechanisms based on matrices," 1955.